

Graph Engineering and Advanced Structures

Lecture 7 | Traversal, Connectivity, MST & Shortest Paths

Algorithm Engineering

July 8, 2026

- 1 Graph Primitives and Connectivity
- 2 Minimum Spanning Trees (MST)
- 3 Single-Source Shortest Paths (SSSP)
- 4 Priority Queues & Advanced Data Structures
- 5 Fibonacci Heaps

Learning Outcomes

By the end of this lecture you should be able to:

- Implement and reason about **BFS**, **DFS**, and **topological sort**.
- Build a **Minimum Spanning Tree** using Kruskal's or Prim's algorithm.
- Compute **single-source shortest paths** with Bellman–Ford and Dijkstra.
- Judge how the choice of **heap** (binary vs. Fibonacci) changes real-world performance.

I. Graph Primitives and Connectivity

20 minutes

Definition: Graph

A graph $G = (V, E)$ consists of a vertex set V and an edge set E where each edge (u, v) represents a connectivity between two vertex u and v . We write $n = |V|$, $m = |E|$.

Adjacency List

- Array of $|V|$ lists; list i holds neighbours of i .
- **Space:** $O(V + E)$ efficient for **sparse** graphs.
- Checking if $(u, v) \in E$: $O(\deg(u))$.

Adjacency Matrix

- $|V| \times |V|$ 0/1 (or weight) matrix.
- **Space:** $O(V^2)$ good for **dense** graphs.
- Checking if $(u, v) \in E$: $O(1)$.

Engineering Rule of Thumb

Use **adjacency lists** when $E \ll V^2$. Use a **matrix** only when $O(1)$ edge lookups matter more than memory, e.g. very dense or small graphs.

Idea

BFS explores vertices level by level using a FIFO queue, ensuring that all nodes at a distance of k edges from the starting source are fully processed before moving outward to any nodes at a distance of $k + 1$, which guarantees finding the shortest path in unweighted graphs.

Algorithm: $\text{BFS}(G, s)$

```
1: color[v] ← WHITE, d[v] ← ∞ for all v ∈ V
2: color[s] ← GRAY, d[s] ← 0, ENQUEUE(Q, s)
3: while Q ≠ ∅ do
4:   u ← DEQUEUE(Q)
5:   for each v ∈ Adj[u] do
6:     if color[v] = WHITE then
7:       color[v] ← GRAY, d[v] ← d[u] + 1, π[v] ← u
8:       ENQUEUE(Q, v)
9:     end if
10:  end for
11:  color[u] ← BLACK
12: end while
```

Outputs

- $d[v]$: the **shortest path distance** from s to v , measured in **number of edges** (unweighted graphs only).
- $\pi[v]$: predecessor pointers, together forming a **Breadth-First Tree** rooted at s .

Key Idea

BFS gives shortest paths *only* when all edges have equal weight. For weighted graphs you need Dijkstra or Bellman–Ford.

Complexity

$O(V + E)$ with adjacency lists.

Each vertex enqueued/dequeued once; each edge examined at most twice (once per endpoint in undirected case).

Idea

DFS explores as **deep** as possible along each branch before **backtracking**, naturally implemented with recursion (an implicit **stack**).

Algorithm: DFS-VISIT(G, u)

```
1:  $time \leftarrow time + 1$ ;  $d[u] \leftarrow time$  ▷ discovery time
2:  $color[u] \leftarrow GRAY$ 
3: for each  $v \in Adj[u]$  do
4:   if  $color[v] = WHITE$  then
5:      $\pi[v] \leftarrow u$ ; DFS-VISIT( $G, v$ )
6:   end if
7: end for
8:  $color[u] \leftarrow BLACK$ 
9:  $time \leftarrow time + 1$ ;  $f[u] \leftarrow time$  ▷ finish time
```

Timestamps (exam favourite!)

- $d[v]$: **discovery** time.
- $f[v]$: **finish** time.
- **Parenthesis theorem**: for any u, v , the intervals $[d[u], f[u]]$ and $[d[v], f[v]]$ are either disjoint or one nests entirely inside the other.

Complexity

$O(V + E)$ — each vertex visited once, each edge examined once.

Edge classification via colour of v when (u, v) explored:

- **Tree edge**: v is WHITE.
- **Back edge**: v is GRAY (ancestor) $\Rightarrow$ indicates a **cycle**.
- **Forward edge**: v is BLACK, $d[u] < d[v]$ (descendant).
- **Cross edge**: v is BLACK, $d[u] > d[v]$.

Topological Sort in DAGs

Definition

A **topological sort** of a DAG $G = (V, E)$ is a linear ordering of vertices such that for every edge (u, v) , u appears before v .

Condition

Only defined for **Directed Acyclic Graphs**. A directed graph has a topological sort $\iff$ it is acyclic $\iff$ DFS on it produces **no back edges**.

Algorithm

- 1: Call $\text{DFS}(G)$ to compute finish times $f[v]$ for all v
- 2: As each vertex finishes, insert it onto the **front** of a linked list
- 3: **Return** the linked list (vertices in decreasing order of $f[v]$)

Complexity: $O(V + E)$, inherited directly from DFS.

Use case: scheduling, build systems, course prerequisites.

II. Minimum Spanning Trees

30 minutes

Minimum Spanning Trees — Definition

Definition: MST

Given a connected, undirected, weighted graph $G = (V, E, w)$, a **Minimum Spanning Tree** is an acyclic subset $T \subseteq E$ that connects all vertices ($|T| = |V| - 1$) and minimises $w(T) = \sum_{(u,v) \in T} w(u, v)$.

Two Structural Properties Every Correct MST Algorithm Relies On

- **Cut property:** for any cut $(S, V - S)$, the minimum-weight edge crossing it belongs to *some* MST.
- **Cycle property:** the maximum-weight edge on any cycle is *not* in the MST (assuming distinct weights, ties broken consistently).

Both Kruskal's and Prim's algorithms are **greedy** they are correct precisely because of the cut property.

Strategy

Sort all edges by weight; repeatedly add the **globally cheapest** remaining edge, as long as it does not create a cycle.

Algorithm: $\text{Kruskal}(G, w)$

```
1:  $T \leftarrow \emptyset$ 
2: sort edges of  $E$  into non-decreasing order by weight
3: for each vertex  $v \in V$  do
4:    $\text{MAKE-SET}(v)$ 
5: end for
6: for each edge  $(u, v)$  in sorted order do
7:   if  $\text{FIND-SET}(u) \neq \text{FIND-SET}(v)$  then
8:      $T \leftarrow T \cup \{(u, v)\}$ 
9:      $\text{UNION}(u, v)$ 
10:  end if
11: end for
12: return  $T$ 
```

Data Structure: Disjoint-Set (Union-Find)

- MAKE-SET, FIND-SET, UNION detect whether adding an edge closes a cycle.
- **Path compression** + **union by rank** give near-constant amortised time: $O(\alpha(n))$ per operation.

Complexity

$O(E \lg V)$ — **dominated by sorting** the edges. Union-Find operations cost only $O(E \alpha(V))$, negligible in comparison.

Exam Tip

If edges arrive **pre-sorted** (or use bucket/radix sort on small integer weights), Kruskal can run in $O(E \alpha(V)) \approx O(E)$.

Strategy

Grow a **single tree** from an arbitrary root r . At each step, greedily attach the **cheapest edge** connecting the current tree to a vertex outside it.

Algorithm: Prim(G, w, r)

```
1:  $key[v] \leftarrow \infty$  for all  $v$ ;  $key[r] \leftarrow 0$ ;  $\pi[v] \leftarrow \text{NIL}$ 
2:  $Q \leftarrow \text{MIN-PRIORITY-QUEUE}$  of all  $v \in V$ , keyed by  $key[v]$ 
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
5:   for each  $v \in \text{Adj}[u]$  do
6:     if  $v \in Q$  and  $w(u, v) < key[v]$  then
7:        $\pi[v] \leftarrow u$ ;  $key[v] \leftarrow w(u, v)$ 
8:     end if
9:   end for
10: end while
```

▷ DECREASE-KEY

Data Structure: Min-Priority Queue on the "fringe" vertices adjacent to the tree, keyed by the cheapest known connecting edge weight.

Complexity

- **Binary heap:** $O(E \lg V)$ (V EXTRACT-MINs at $O(\lg V)$ + E DECREASE-KEYs at $O(\lg V)$).
- **Fibonacci heap:** $O(E + V \lg V)$ (DECREASE-KEY becomes $O(1)$ amortised).

Kruskal vs. Prim — Which to Choose?

- **Sparse graph** ($E \approx V$): Kruskal, dominated by an easy sort.
- **Dense graph** ($E \approx V^2$): Prim with Fibonacci heap wins, since $O(E + V \lg V)$ beats $O(E \lg V)$.
- Need edges sorted globally already? Kruskal is trivial to code. Need to grow incrementally from a fixed root (e.g. streaming)? Prim.

III. Single-Source Shortest Paths

30 minutes

Definition

For each vertex v , maintain $d[v]$: the current best **upper bound** estimate of $\delta(s, v)$, the true shortest-path weight from s to v . Initially $d[s] = 0$, $d[v] = \infty$ otherwise.

RELAX(u, v, w)

```
1: if  $d[v] > d[u] + w(u, v)$  then
2:    $d[v] \leftarrow d[u] + w(u, v)$ 
3:    $\pi[v] \leftarrow u$ 
4: end if
```

Key Idea

Every shortest-path algorithm (BFS, Bellman–Ford, Dijkstra, DAG-shortest-paths) is just **relaxation applied in a different order**. The order is what determines correctness and running time.

Strategy

Relax **every** edge, $|V| - 1$ times in total. After $|V| - 1$ rounds, all shortest-path estimates are correct (if no negative cycle reachable from s).

Algorithm: Bellman-Ford(G, w, s)

```
1: INITIALIZE-SINGLE-SOURCE( $G, s$ )
2: for  $i \leftarrow 1$  to  $|V| - 1$  do
3:   for each edge  $(u, v) \in E$  do
4:     RELAX( $u, v, w$ )
5:   end for
6: end for
7: for each edge  $(u, v) \in E$  do
8:   if  $d[v] > d[u] + w(u, v)$  then
9:     return FALSE
10:  end if
11: end for
12: return TRUE
```

▷ negative-cycle check

Why $|V| - 1$ passes? A shortest simple path has at most $|V| - 1$ edges. Each full pass over E guarantees the shortest path with i edges is found by round i .

Key Advantage

Correctly handles **negative-weight edges** and can **detect** negative-weight cycles reachable from s (the extra check pass).

Complexity

$$O(V \cdot E)$$

$|V| - 1$ passes, each examining all $|E|$ edges.

Strategy

Greedy: maintain a set S of vertices whose final shortest-path weight is known. Repeatedly extract $u \in V - S$ with **minimum** $d[u]$, add it to S , and relax its outgoing edges.

Algorithm: Dijkstra(G, w, s)

```
1: INITIALIZE-SINGLE-SOURCE( $G, s$ );  $S \leftarrow \emptyset$ ;  $Q \leftarrow V$  ▷ min-PQ keyed on  $d$ 
2: while  $Q \neq \emptyset$  do
3:    $u \leftarrow \text{EXTRACT-MIN}(Q)$ 
4:    $S \leftarrow S \cup \{u\}$ 
5:   for each  $v \in \text{Adj}[u]$  do
6:     RELAX( $u, v, w$ ) ▷ updates  $Q$  via DECREASE-KEY
7:   end for
8: end while
```

Constraint

Requires **non-negative edge weights** otherwise a vertex already in S could later be improved, breaking the greedy argument.

Complexity by Priority-Queue Choice

- **Array (no heap):** $O(V^2)$ — fine for dense graphs, no heap overhead.
- **Binary heap:** $O(E \lg V)$ — V EXTRACT-MINs + E DECREASE-KEYs, each $O(\lg V)$.
- **Fibonacci heap:** $O(V \lg V + E)$ — DECREASE-KEY is $O(1)$ amortised, so the E term drops the $\lg V$ factor.

Where Fibonacci Wins

When $E \gg V$ (**dense graphs**), $O(V \lg V + E)$ is asymptotically better than $O(E \lg V)$, since the expensive $\lg V$ factor no longer multiplies the large E term.

	Bellman–Ford	Dijkstra
Edge weights	negative allowed	non-negative only
Negative cycles	detects them	undefined behaviour
Strategy	dynamic programming	greedy
Complexity	$O(VE)$	$O(E \lg V)$ / $O(V \lg V + E)$
When to use	currency arbitrage, unreliable weights	road networks, non-negative costs

Key Idea

Rule of thumb: **always prefer Dijkstra** when weights are guaranteed non-negative it is asymptotically faster. Fall back to Bellman–Ford only when negative weights are possible.

IV. Priority Queues & Advanced Data Structures

20 minutes

Operation	Binary Heap	Fibonacci Heap
INSERT	$O(\lg n)$	$O(1)$ amortised
EXTRACT-MIN	$O(\lg n)$	$O(\lg n)$ amortised
DECREASE-KEY	$O(\lg n)$	$O(1)$ amortised
UNION	$O(n)$	$O(1)$

Structural Difference

A binary heap is a single compact array-based tree. A Fibonacci heap is a **collection of heap-ordered trees** with lazy consolidation, enabling fast DECREASE-KEY by cutting nodes without immediate restructuring.

Fibonacci Heaps

Deep Dive

Fibonacci Heap — Structure

Definition

A **Fibonacci heap** H is a collection of **min-heap-ordered trees**: for every node x , $key[parent(x)] \leq key[x]$. The trees are linked into a **circular doubly-linked root list**, and H keeps a pointer $min[H]$ to the smallest root.

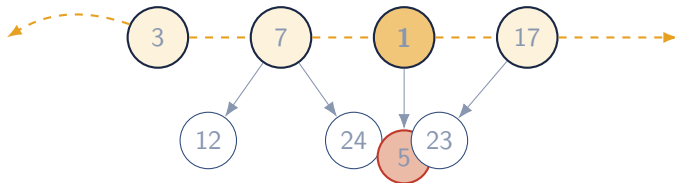
Per-node fields:

- $key[x]$ — the value used for ordering.
- $degree[x]$ — number of children of x .
- $mark[x]$ — boolean: has x lost a child since it last became a child of another node?
- $p[x]$, $child[x]$ — pointer to parent, pointer to one child.
- $left[x]$, $right[x]$ — circular doubly-linked list of x 's siblings.

Why This Design?

Doubly-linked circular lists give $O(1)$ **splice** (cut/concatenate) operations — the key enabler behind fast UNION and DECREASE-KEY.

Fibonacci Heap — Visual Structure



- **Amber-filled root** = $\min[H]$ (here, key 1).
- **Dashed line** = the circular root list.
- **Red node** = a **marked** node (has already lost one child).

Definition: Potential

For a Fibonacci heap H , define

$$\Phi(H) = t(H) + 2m(H)$$

where $t(H)$ = number of trees in the root list, and $m(H)$ = number of marked, non-root nodes.

How It Is Used

Amortized cost = actual cost + $\Delta\Phi$. Operations that increase $t(H)$ (like INSERT) pay a small amortized cost; operations that *decrease* $t(H)$ a lot (like the consolidation in EXTRACT-MIN) can be expensive in actual work but the potential drop **pays for it**, keeping the amortized cost low.

This single trick is why Fibonacci heaps achieve $O(1)$ amortized INSERT, UNION, DECREASE-KEY, and $O(\lg n)$ amortized EXTRACT-MIN.

INSERT(H, x)

```
1:  $degree[x] \leftarrow 0$ ;  $p[x] \leftarrow \text{NIL}$ ;  $child[x] \leftarrow \text{NIL}$ ;  $mark[x] \leftarrow \text{FALSE}$ 
2: add  $x$  to the root list as a single-node tree
3: if  $min[H] = \text{NIL}$  or  $key[x] < key[min[H]]$  then
4:    $min[H] \leftarrow x$ 
5: end if
6:  $n[H] \leftarrow n[H] + 1$ 
```

Complexity

Actual: $O(1)$ (splice into a circular list). **Amortized:** $O(1)$ — $t(H)$ increases by exactly 1, so $\Delta\Phi = 1$.

MINIMUM(H): simply return $min[H]$ — $O(1)$, no structural change at all.

UNION(H_1, H_2)

- 1: concatenate the root lists of H_1 and H_2 into a single circular list
- 2: $\min[H] \leftarrow \min[H_1]$ if $\text{key}[\min[H_1]] \leq \text{key}[\min[H_2]]$, else $\min[H_2]$
- 3: $n[H] \leftarrow n[H_1] + n[H_2]$

Why $O(1)$?

Splicing two circular doubly-linked lists together is $O(1)$ — **no per-node work is needed**. This is precisely where Fibonacci heaps beat binary heaps, whose UNION costs $O(n)$ (rebuilding the array-based heap).

EXTRACT-MIN(H)

```
1:  $z \leftarrow \text{min}[H]$ 
2: if  $z \neq \text{NIL}$  then
3:   for each child  $x$  of  $z$  do
4:     add  $x$  to the root list;  $p[x] \leftarrow \text{NIL}$ 
5:   end for
6:   remove  $z$  from the root list
7:   if  $z = \text{right}[z]$  then  $\text{min}[H] \leftarrow \text{NIL}$ 
8:   else  $\text{min}[H] \leftarrow \text{right}[z]$ ; CONSOLIDATE( $H$ )
9:   end if
10:   $n[H] \leftarrow n[H] - 1$ 
11: end if
12: return  $z$ 
```

Idea: the minimum's children have no parent to answer to once z is gone — promote them all to the root list, then clean up with CONSOLIDATE.

Goal

After promoting z 's children, the root list may have **many trees of the same degree**. Consolidate merges them so that **no two roots share a degree**, bounding the number of trees by $O(\lg n)$.

CONSOLIDATE(H)

```

1: let  $A[0..D(n)]$  be an array of tree pointers, all initialised to NIL
2: for each root  $w$  in the root list do
3:    $x \leftarrow w$ ;  $d \leftarrow \text{degree}[x]$ 
4:   while  $A[d] \neq \text{NIL}$  do
5:      $y \leftarrow A[d]$                                 ▷ another root with the same degree
6:     if  $\text{key}[x] > \text{key}[y]$  then swap  $x \leftrightarrow y$ 
7:     end if
8:     LINK( $y, x$ )                                       ▷ make  $y$  a child of  $x$ 
9:      $A[d] \leftarrow \text{NIL}$ ;  $d \leftarrow d + 1$ 
10:  end while
11:   $A[d] \leftarrow x$ 
12: end for
13: rebuild the root list from  $A$ ; recompute  $\min[H]$ 

```

Actual cost = $O(D(n) + t(H))$

- $D(n) = O(\lg n)$: max possible degree of any node in an n -node Fibonacci heap.
- $t(H)$: trees before consolidating (could be large after many inserts).

Potential drop pays for it

- Consolidation can reduce $t(H)$ down to at most $D(n) + 1$.
- $\Delta\Phi \approx -(t(H) - D(n))$, which **cancels** the large $t(H)$ term in the actual cost.

Result

Amortized cost of EXTRACT-MIN = $O(\lg n)$.

Goal

Decrease $key[x]$ to a new, smaller value, and restore the min-heap property **without** an expensive re-heapify.

DECREASE-KEY(H, x, k)

```
1: if  $k > key[x]$  then error "new key is greater"
2: end if
3:  $key[x] \leftarrow k$ 
4:  $y \leftarrow p[x]$ 
5: if  $y \neq \text{NIL}$  and  $key[x] < key[y]$  then
6:   CUT( $H, x, y$ )
7:   CASCADING-CUT( $H, y$ )
8: end if
9: if  $key[x] < key[\min[H]]$  then  $\min[H] \leftarrow x$ 
10: end if
```

Key idea: if decreasing x 's key violates heap order with its parent y , simply **cut** x **free** and

CUT(H, x, y)

- 1: remove x from the child list of y ; $degree[y] \leftarrow degree[y] - 1$
- 2: add x to the root list of H ; $p[x] \leftarrow \text{NIL}$; $mark[x] \leftarrow \text{FALSE}$

CASCADING-CUT(H, y)

- 1: $z \leftarrow p[y]$
- 2: **if** $z \neq \text{NIL}$ **then**
- 3: **if** $mark[y] = \text{FALSE}$ **then** $mark[y] \leftarrow \text{TRUE}$
- 4: **else** CUT(H, y, z); CASCADING-CUT(H, z)
- 5: **end if**
- 6: **end if**

The Marking Rule — Why It Bounds the Damage

The Rule

A node is **marked** the **first time** it loses a child after becoming someone's child itself. If it loses a **second** child, it is immediately **cut** and becomes a root — and the cut **cascades upward** to its own parent.

- This enforces: **every node loses at most one child** before being cut loose itself.
- It is exactly what keeps subtree sizes large enough to guarantee $D(n) = O(\lg n)$ — the Fibonacci-number-growth property the structure is named after.

Amortized Cost of DECREASE-KEY

A cascading cut of length c does $O(c)$ actual work but **decreases** $t(H)$ by... no — it **increases** $t(H)$ by c while **decreasing** $m(H)$ by at least $c - 1$ (each cascaded node becomes unmarked). Since $\Phi = t(H) + 2m(H)$, the drop in $2m(H)$ cancels the cost of the cuts, leaving $O(1)$ **amortized** overall.

DELETE(H, x)

- 1: DECREASE-KEY($H, x, -\infty$)
- 2: EXTRACT-MIN(H)

▷ make x the unique minimum
▷ remove it

Complexity

Amortized $O(\lg n)$ — dominated entirely by the EXTRACT-MIN call, since DECREASE-KEY is $O(1)$ amortized.

Design Lesson

This is a recurring theme in algorithm engineering: **build a rarely-needed operation out of primitives you have already paid to make fast**, instead of writing new logic.

Fibonacci Heap — Complexity Summary

Operation	Actual Cost	Amortized Cost
MAKE-HEAP	$O(1)$	$O(1)$
INSERT	$O(1)$	$O(1)$
MINIMUM	$O(1)$	$O(1)$
UNION	$O(1)$	$O(1)$
EXTRACT-MIN	$O(D(n) + t(H))$	$O(\lg n)$
DECREASE-KEY	$O(c)$ (cascade length)	$O(1)$
DELETE	$O(D(n) + t(H))$	$O(\lg n)$

Exam Tip

The two operations to remember cold: DECREASE-KEY is $O(1)$ amortised (this is the whole point of Fibonacci heaps, and what makes Dijkstra/Prim run in $O(E + V \lg V)$), while EXTRACT-MIN stays at $O(\lg n)$, same order as a binary heap.

Engineering Context

Fibonacci heaps are **asymptotically superior**, yet binary heaps are **often faster in practice**. Why?

- **High constant factors:** Fibonacci heap operations involve pointer-heavy structures (circular doubly-linked lists, lazy consolidation bookkeeping).
- **Cache locality:** a binary heap's array layout is contiguous in memory far more cache-friendly than scattered tree nodes.
- **Implementation simplicity:** binary heaps are easy to implement correctly; Fibonacci heaps are notoriously fiddly (many bugs live in the CASCADING-CUT step).

Takeaway

Use Fibonacci heaps only when the graph is **provably dense enough** ($E \gg V \lg V$) that the asymptotic gain outweighs constant-factor overhead e.g. very large, dense weighted graphs in research-grade shortest-path solvers.

Case Study Questions

- Dijkstra vs. Bellman–Ford? → Are negative weights possible?
- Prim vs. Kruskal? → Is the graph sparse or dense? Do edges arrive pre-sorted?
- Binary vs. Fibonacci heap? → Is E large relative to $V \lg V$, and does implementation complexity matter?

One-Line Mental Model

BFS/DFS explore; **MST algorithms** connect cheaply; **SSSP algorithms** relax distances; **heaps** decide how fast the greedy choice can be found.

Exam-Ready Recap: All Complexities at a Glance

Algorithm	Complexity	Requires
BFS	$O(V + E)$	unweighted
DFS	$O(V + E)$	any
Topological Sort	$O(V + E)$	DAG
Kruskal (MST)	$O(E \lg V)$	edge sort
Prim (binary heap)	$O(E \lg V)$	—
Prim (Fibonacci heap)	$O(E + V \lg V)$	—
Bellman–Ford	$O(VE)$	any weights
Dijkstra (binary heap)	$O(E \lg V)$	$w \geq 0$
Dijkstra (Fibonacci heap)	$O(V \lg V + E)$	$w \geq 0$

Before Next Lecture

- **CLRS** (*Introduction to Algorithms*): Chapters 20 (Elementary Graph Algorithms), 21 (MST), 22 (Single-Source Shortest Paths).
- **Levitin** (*Design and Analysis of Algorithms*): Chapter 9.

Questions?